

PLAN DE TRABAJO COLABORATIVO

Práctica 3: Programación Multi-hilo - Smart Parking

PROYECTO DE UN DÍA

Universidad Carlos III de Madrid

Sistemas Operativos - Curso 2025/2026

Equipo: 3 Desarrolladores (SINCRONIZADOS)

Índice de Contenidos

1. Resumen Ejecutivo
2. Estructura del Proyecto
3. División de Tareas
4. Cronograma del Día
5. Flujo de Trabajo Git
6. Sincronización B+C (CRÍTICO)
7. Requerimientos Críticos
8. Checklist Final

1. Resumen Ejecutivo

TODO EN UN DÍA

- 3 personas trabajan en paralelo en el MISMO archivo (parking_manager.c)
- Máxima coordinación entre PERSON B y PERSON C
- PERSON A trabaja independientemente en queue.c
- Entrega: Al final del día

Objetivos del Plan

- Minimizar conflictos de merge (B y C coordinados)
- Compilación sin warnings
- Funcionalidad correcta en un push
- Memoria completada

Estrategia

- PERSON A: queue.c (independiente)
- PERSON B y C: parking_manager.c (juntos, en pares, máxima comunicación)
- Commits muy frecuentes
- Push directos a main
- Sincronización en tiempo real

2. Estructura del Proyecto

Archivos a Entregar

- queue.h - Header (NO TOCAR)
- queue.c - PERSON A
- parking_manager.c - PERSON B + C (JUNTOS)
- Makefile - NO TOCAR
- autores.txt - YA EXISTE

3. División de Tareas

PERSONA A: Gestor de Colas Circulares

Archivo: queue.c (SOLO, independiente)

9. queue_init(int size)
10. queue_put() con sincronización
11. queue_get() con sincronización
12. queue_empty() y queue_full()
13. queue_destroy()

VENTAJA: Trabaja SOLO, sin coordinación con B ni C

PERSONA B + C: Main y Control de Estacionamiento

Archivo: parking_manager.c (COMPARTIDO - MÁXIMA COORDINACIÓN)

PERSON B: Sección Lot (líneas aprox 10-50)

- Estructura Lot
- lot_claim_space()
- lot_release_space()
- Inicialización de Lot

PERSON C: main() y Threads (líneas 50+)

- main()
- entry_lane_thread()
- exit_gate_thread()
- CSV parsing



CRÍTICO: B y C DEBEN ESTAR JUNTOS FRENTE A UN PC

- B implementa Lot, C lo va usando en main
- Cada 10 minutos: pull + compilar + testear
- Si B termina Lot: C puede continuar
- Si C necesita cambios en Lot: B ajusta
- Comunicación verbalmente

4. Cronograma del Día (Horario)

Hora	PERSON A	PERSON B	PERSON C
09:00	Diseño queue.c	Diseño Lot (JUNTOS)	Diseño main (JUNTOS)
10:00	Impl queue funciones	Impl Lot structs+mutex	Impl main CSV parsing
11:30	Impl queue completo	Impl lot_claim	Impl threads entrada
13:00	ALMUERZO	ALMUERZO	ALMUERZO
14:00	Testing queue	Impl lot_release	Impl threads salida
15:30	Push queue.c	Testing Lot (JUNTOS)	Testing main (JUNTOS)
16:00	-	Push Lot (JUNTOS)	Push main (JUNTOS)
16:30	-	GUERNIKA	GUERNIKA
17:00	-	Checker Python	Checker Python
17:30	Memoria	Memoria	Memoria
18:30	ENTREGA	ENTREGA	ENTREGA

Notas del Cronograma

- PERSON A trabaja INDEPENDIENTE todo el día
- PERSON B y C: MÁXIMA coordinación, posiblemente en el MISMO PC
- Cada hora: compilar 'make clean && make'
- 16:30: Probar en Guernika (puede ser remoto si hace falta)
- 17:00: Ejecutar checker_ssoo_p3.py
- 17:30: Todos hacen memoria de forma paralela
- 18:30: ZIP + Entrega

5. Flujo de Trabajo Git (UN DÍA)

Configuración Inicial (TODOS)

```
git clone <repository>
git pull origin main
```

Durante el Día

PERSON A:

- Modifica SOLO queue.c
- Commits cada función completada
- git push origin main

PERSON B y C:

- Antes de empezar: git pull origin main
- Cada 30 min: git pull origin main (traer cambios del otro)
- Commits muy frecuentes en parking_manager.c
- Si hay conflicto: resolver JUNTOS en el mismo PC
- Push cuando: Lot lista → C puede continuar
- Push cuando: main + threads listos → B puede refinar Lot

Ejemplo de Sincronización B+C

14. B y C se sientan juntos frente al mismo PC
15. B implementa: struct Lot, mutexes, cond variables
16. 10:30 - B pushea: struct Lot lista
17. C hace git pull, ve struct Lot
18. C empieza a usar parking_lot en main
19. B implementa: lot_claim_space
20. 11:15 - B pushea: lot_claim_space lista
21. C hace git pull, usa lot_claim_space en threads
22. Y así sucesivamente...

6. Sincronización B+C (CRÍTICO)

 **MÁXIMA IMPORTANCIA** 

B y C DEBEN ESTAR JUNTOS

- Preferiblemente en el MISMO asiento
- Mismo PC si es posible
- Si no: pantalla compartida en Zoom/Teams
- Comunicación verbal TODO el tiempo

Secciones Claras en parking_manager.c

```
// ===== PERSON B - LOT SECTION =====
typedef struct { ... } Lot;
void lot_claim_space(Lot *l) { ... }
void lot_release_space(Lot *l) { ... }
// ===== END PERSON B - LOT SECTION =====

// ===== PERSON C - MAIN + THREADS =====
int main() { ... }
void *entry_lane_thread() { ... }
void *exit_gate_thread() { ... }
// ===== END PERSON C - MAIN + THREADS =====
```

Protocolo de Sincronización

23. B: "Voy a hacer struct Lot y mutex", C espera
24. B: "Listo struct Lot", pushea
25. C: git pull, ve struct, "OK, uso Lot en main"
26. B: "Implementando lot_claim_space"
27. C: "Espero, no necesito lot_claim aún", sigue con queue_put
28. B: "lot_claim_space lista", pushea
29. C: git pull, "Bien, lo uso en entry_lane_thread"
30. Y así...

Si Hay Conflicto

- NO PÁNICO
- Los dos abren parking_manager.c
- Buscan <<<<<< HEAD

- Uno mantiene sección B (Lot)
- Otro mantiene sección C (main + threads)
- git add, git commit, git push

7. Requerimientos Críticos

Prohibido (Fallo automático)

31. ✗ fopen, fread, fwrite, fclose
32. ✗ goto
33. ✗ Compilar con warnings
34. ✗ Modificar Makefile o queue.h

Obligatorio

35. ✓ open(), read(), close()
36. ✓ perror si error → return -1
37. ✓ Validar CSV
38. ✓ Mutex + condition variables
39. ✓ Prioridades: 0 > 1 > 2
40. ✓ Outputs exactos
41. ✓ Guernika OK

Formatos de Salida

[OK][parking_manager] Finishing
 [LANE <lane>][GATE <gate>] Waiting / Car / Ending
 [EXIT <gate>] CAR <plate> / Ending
 [INFO] LOT CLAIMED <n>/<total> BY <plate>
 [INFO] LOT RELEASED <n>/<total> BY <plate>

8. Checklist Final (16:00h)

Compilación

42. ☐ make clean && make SIN warnings
43. ☐ Ejecutar en Guernika
44. ☐ checker_ssoo_p3.py OK

Código

45. ☐ Queue funciones todas
46. ☐ Lot estructuras y funciones
47. ☐ main() completo
48. ☐ entry_lane_thread
49. ☐ exit_gate_thread
50. ☐ Sincronización correcta
51. ☐ NO fopen/fread/fwrite
52. ☐ Memoria liberada
53. ☐ Comentarios abundantes

Entrega

- 54. ☐ Todos cambios en main
- 55. ☐ ZIP: ss00_p3_NIA1_NIA2.zip
- 56. ☐ autores.txt
- 57. ☐ Memoria 15 páginas máx
- 58. ☐ Subir a Aula Global